FORMATION

Analyse de données avec Python

Niveau intermédiaire, 3 jours

Durée : 3 jours, soit 6 demi journées de 3h30

Public : Développeurs, analystes ou data analysts ayant déjà pratiqué Python

Prérequis : Bases de Python (variables, boucles, fonctions, listes et dictionnaires) et notions élémentaires de statistiques descriptives

Format : Alternance de cours (démonstrations commentées) et de travaux pratiques sur Jupyter Notebook

Groupe : 4 participants, un poste par participant avec Python installé

Objectifs pédagogiques

- Manipuler, nettoyer et transformer des données tabulaires avec pandas.
- Agréger, croiser, combiner et automatiser le traitement de plusieurs sources de données.
- Produire des visualisations statiques et interactives avec matplotlib, seaborn et plotly.
- Structurer un projet Python robuste (dossiers, modules, gestion des erreurs, typage).
- Transformer un notebook d'exploration en outil autonome : script en ligne de commande avec journalisation et configuration.
- Gérer proprement les dépendances d'un projet avec pip et avec uv.

Vue d'ensemble du programme

Jour	Demi journée	Thème	Format
Jour 1	Matin	Environnement de travail (pip, venv), rappel numpy et prise en main de pandas dans Jupyter	Cours + TP
Jour 1	Après midi	Sélection, transformation et nettoyage de données	Cours + TP
Jour 2	Matin	Agrégation, dates et automatisation de plusieurs fichiers	Cours + TP
Jour 2	Après midi	Visualisation avec matplotlib, seaborn et plotly	Cours + TP
Jour 3	Matin	Organisation de projet, code robuste et CLI	Cours + TP
Jour 3	Après midi	Gestion des dépendances (pip, uv) et projet final	Cours + TP fil rouge
Bonus	Si le temps le permet	Polars, DuckDB, Streamlit, interfaces, empaquetage	Découverte + TP bonus

Une section bonus (Polars, DuckDB, Streamlit, empaquetage en exécutable, etc.) est proposée en fin de formation, à dérouler si le temps le permet, en plus des 21 heures du programme principal.

Jour 1 : Manipulation de données avec pandas

Matin : Environnement de travail (pip, venv), rappel numpy et prise en main de pandas *(3h30)*

Cours

- Python intermédiaire utile à l'analyse de données : compréhensions de liste et de dictionnaire, fonctions lambda, gestion des exceptions de base.
- Mise en place de l'environnement de travail : création d'un environnement virtuel avec venv, installation des bibliothèques avec pip, fichier requirements.txt.
- Lancement de Jupyter Notebook et JupyterLab depuis cet environnement, premiers repères dans l'interface.
- Mini rappel numpy (très court) : tableaux (array), opérations vectorisées, différence avec les listes Python.
- Introduction à pandas : la Series et le DataFrame, notion d'index, différences avec les listes et dictionnaires natifs.
- Lecture et écriture de fichiers : CSV, Excel, Parquet, principales options de read_csv.
- Première exploration d'un jeu de données : head, tail, info, describe, dtypes, shape, value_counts.

Mini exercice numpy (15 minutes environ) : quelques opérations vectorisées simples (moyenne, filtrage, comparaison de performance avec une boucle Python classique), avant de passer à pandas.

TP 1. Environnement de travail et première exploration dans un notebook

Objectif : Créer un environnement virtuel avec venv, installer les bibliothèques nécessaires avec pip, lancer Jupyter Notebook, puis charger un jeu de données réel (ventes ou dataset public type transport ou immobilier) et l'explorer directement dans le notebook pour visualiser les résultats au fur et à mesure.

Livrable : Un environnement virtuel fonctionnel avec un fichier requirements.txt, et un notebook d'exploration commentée du jeu de données (dimensions, types, valeurs aberrantes repérées, premières questions à creuser).

Après midi : Sélection, transformation et nettoyage de données *(3h30)*

Cours

- Sélection de données : loc, iloc, filtrage booléen, sélection combinée de lignes et colonnes.
- Transformation : création et suppression de colonnes, apply, map, fonctions vectorisées.
- Nettoyage de texte avec l'accessor .str : espaces, casse, remplacement, extraction de motifs simples.
- Gestion des types : conversion (astype), catégories, types datetime.
- Valeurs manquantes et doublons : détection (isna, duplicated), stratégies de traitement (dropna, fillna, interpolate).
- Tri, renommage de colonnes, réindexation, set_index et reset_index.
- Export au format Parquet : pourquoi ce format compresse et charge plus vite qu'un CSV pour un usage interne.

TP 2. Nettoyage d'un jeu de données

Objectif : Repérer et corriger les valeurs manquantes, les doublons, les types incorrects et les colonnes texte mal formatées du jeu de données du TP1, harmoniser les noms de colonnes.

Livrable : Un dataset propre exporté en CSV et en Parquet (avec comparaison rapide de la taille des deux fichiers), accompagné d'une courte note listant les choix de nettoyage effectués.

Jour 2 : Agrégation, jointures et visualisation

Matin : Agrégation, dates et automatisation de plusieurs fichiers *(3h30)*

Cours

- Agrégation : groupby, agg, fonctions d'agrégation multiples, pivot_table (mention rapide de crosstab).
- Combinaison de sources : concat, merge et join, gestion des clés et des types de jointure.
- Manipulation des dates : type datetime, extraction (année, mois, jour de semaine), resample pour les séries temporelles (mention rapide de rolling et expanding pour les moyennes mobiles).
- Automatiser le traitement de plusieurs fichiers : parcourir un dossier avec pathlib et glob, appliquer le même traitement à chaque fichier trouvé dans une boucle.

TP 3. Construction d'indicateurs consolidés

Objectif : À partir de plusieurs fichiers sources placés dans un même dossier (par exemple un fichier par mois ou par région), parcourir automatiquement le dossier, joindre les tables et calculer des indicateurs agrégés par mois et par région.

Livrable : Une table consolidée exportée (Parquet) contenant les indicateurs calculés, prête à être utilisée pour la visualisation.

Après midi : Visualisation de données *(3h30)*

Cours

- matplotlib : anatomie d'une figure (Figure, Axes), tracés de base, personnalisation (titres, légendes, couleurs, sous-graphiques).
- seaborn : graphiques statistiques prêts à l'emploi (distributions, boîtes à moustaches, corrélations, graphiques catégoriels).
- plotly : graphiques interactifs, info bulles, export au format HTML pour le partage.
- Bonnes pratiques de dataviz : choisir le bon type de graphique, lisibilité, hiérarchie visuelle, éviter les graphiques trompeurs.

TP 4. Tableau de bord d'indicateurs

Objectif : À partir de la table consolidée du TP3, produire un ensemble de 4 à 5 graphiques pertinents (au moins un avec seaborn et un interactif avec plotly).

Livrable : Un notebook de restitution avec les graphiques commentés, et un export HTML du graphique interactif principal.

Jour 3 : Bonnes pratiques et gestion de projet Python

Matin : Organisation de projet, code robuste et CLI *(3h30)*

Cours : structurer un projet et un code robuste

- Passer de pip/venv à uv pour mieux gérer un projet : pourquoi cet outil, initialisation avec uv init, fichier pyproject.toml (approfondi cet après midi).
- Structure type d'un projet data : dossiers data, notebooks, src, tests et outputs, rôle de chacun.
- Notebooks contre scripts : quand privilégier l'un ou l'autre, comment faire évoluer un notebook vers du code réutilisable.
- Modularisation du code : extraire des fonctions dans des modules Python, les importer depuis un notebook.
- Type hints (annotations de type) pour documenter et sécuriser les fonctions, mention rapide de mypy.
- Exceptions personnalisées : créer ses propres classes d'erreurs pour un code plus robuste et plus lisible.
- Introduction aux tests avec pytest et à la qualité de code avec un outil comme ruff (lint et formatage).

Cours : transformer un script en outil autonome

- Construire une interface en ligne de commande avec argparse : paramètres, options, valeurs par défaut.
- Remplacer les print par le module logging : niveaux de log, écriture dans un fichier, suivi de l'exécution d'un script autonome.
- Séparer la configuration du code : fichier .env ou config au format YAML plutôt que des chemins ou paramètres codés en dur.

TP 5. D'un notebook à un script en ligne de commande

Objectif : Reprendre le notebook nettoyé des TP précédents, extraire les fonctions dans un module Python typé, gérer les erreurs avec des exceptions dédiées, puis exposer le tout comme un script exécutable en ligne de commande avec des paramètres, des logs et un fichier de configuration.

Livrable : Un mini projet structuré (module Python + script CLI) exécutable en ligne de commande, avec journalisation et configuration externalisée, sans erreur de bout en bout.

Après midi : Gestion des dépendances et projet final *(3h30)*

Cours

- Retour sur les limites de pip et venv pour un projet qui grossit (résolution de dépendances lente, pas de fichier de verrouillage natif).
- Approfondissement de uv découvert ce matin : commandes courantes, uv add pour ajouter une dépendance, uv sync pour installer l'environnement, uv run pour exécuter un script ou un notebook dans l'environnement du projet.
- Reproductibilité : fichier de verrouillage des versions (uv.lock), différence entre dépendances de développement et de production.
- Finitions d'un projet partageable : README, .gitignore, organisation finale des dossiers.

TP 6. Projet fil rouge

Objectif : Créer un projet complet avec uv (structure de dossiers, dépendances déclarées dans pyproject.toml), comprenant un notebook d'analyse s'appuyant sur les données des jours précédents et un script exécutable produisant un rapport ou un graphique final.

Livrable : Un projet Python autonome et reproductible, restitué en fin de formation devant le groupe (5 minutes par participant ou binôme).

Bonus : pour aller plus loin (si le temps le permet)

Ce module n'est pas comptabilisé dans les 21 heures du programme principal. Il est proposé en complément si le rythme du groupe le permet, sous forme de découvertes rapides et, pour certains points, de TP bonus courts.

Données volumineuses : au delà de pandas

- Polars : une alternative à pandas bien plus rapide sur de gros volumes, avec une syntaxe proche, donc une transition facile.
- DuckDB : exécuter des requêtes SQL directement sur des fichiers CSV ou Parquet, sans monter de base de données.
- Spark (pyspark) : mention rapide, pour situer l'outil quand les données sont distribuées sur plusieurs machines.

Restituer une analyse sans notebook

- Streamlit : construire une petite application web interactive (filtres, graphiques) à partir d'un script Python, en quelques lignes.

TP Bonus A. Mini tableau de bord Streamlit

Objectif : Reprendre les indicateurs et graphiques produits dans les TP précédents et les exposer dans une application Streamlit avec au moins un filtre interactif.

Livrable : Une application Streamlit fonctionnelle, lancée en local avec streamlit run.

Interfaces graphiques de bureau

- tkinter : inclus dans la bibliothèque standard, permet de créer une interface simple (fenêtres, boutons, champs) sans rien installer.
- Mention rapide de PyQt et PySide comme alternatives plus riches si le besoin dépasse tkinter.

Empaqueter un script en exécutable autonome

- PyInstaller : transforme un script en .exe (Windows), en binaire Mac ou Linux, sans que l'utilisateur final ait besoin d'installer Python.
- Point d'attention : l'empaquetage se fait sur le système cible (un .exe Windows se génère sur Windows, un binaire Mac sur Mac), ce n'est pas une seule commande valable partout.

TP Bonus B. Empaqueter le script en exécutable

Objectif : Reprendre le script en ligne de commande du TP5 et l'empaqueter avec PyInstaller pour obtenir un exécutable autonome.

Livrable : Un exécutable qui fonctionne sur la machine du participant sans environnement Python installé.

De belles interfaces en ligne de commande

- Rich : mise en forme colorée, tableaux et barres de progression dans le terminal.
- Typer (ou Click) : construire une CLI plus agréable qu'avec argparse seul, en s'appuyant sur les type hints déjà vus le matin.

Autres notions de Python intermédiaire

- Dataclasses : simplifier la création de classes qui servent surtout à stocker des données.
- Décorateurs : notion clé du langage, illustrée par un exemple simple (mesure du temps d'exécution d'une fonction, ou logging automatique).
- Gestionnaires de contexte (with, module contextlib) : comprendre ce qui se passe réellement derrière l'ouverture d'un fichier.
- itertools et functools : quelques fonctions utiles (groupby, reduce, partial) pour aller au delà des boucles classiques.

Annexes

Modalités pédagogiques et évaluation

- Chaque demi journée alterne apports théoriques courts et mise en pratique immédiate.
- Évaluation finale par un QCM.
- Support de cours et corrigés des TP remis aux participants à l'issue de la formation.

Matériel et prérequis techniques

- Un ordinateur par participant
- Accès internet
- Accès à github.com
- Accès à https://versoml.com/ e tout les sous réseaux